

Blueprint Smart Hidroponic Berbasis IoT (Internet Of Things)



1. Ringkasan Fitur Sistem

- Monitoring secara realtime tumbuhan hidroponik dengan menampilkan nilai (suhu air, PH, PPM, Ketinggian air).
- Kontroling pompa air secara otomatis
- Kontroling nutrisi a dan nutrisi b secara otomatis.

2. Bill Of Material (BOM)

Tabel 1. 1 Kebutuhan Komponen Sitem Smart Hidroponic

No	Komponen	Rekomendasi	QTY	Catatan
1	Mikrokontroler	ESP-32S	1	Miktokontroler mempunyai 2 chip.
2	Sensor PH	Sensor PH	1	Mengukur nilai PH pada tandon.
3	Sensor PPM	Sensor TDS	1	Mengukur PPM dialiran air.
4	Sensor Ketinggian Air	Water Level	1	Mengukur volume air pada tandon
5	Relay 4 channel		1	
6	Button	Push Button	3	Untuk menyala relay secara fisik.
7	Sensor Suhu Air	DS18B20	1	Tahan terhadap air
8	Power Supplay	24 Volt	1	Mengubah arus ac menjadi DC dan

				digunakan sebagai sumber sistem.
9	Kabel Tunggal	AWG 28	5 Meter	Menghubungkan antara sensor dan mikrokontroler.
10	Penel	Ukuran 17 cm X 20 cm X 12 cm	1	Melindungi sistem dari cuaca lingkungan dan air.

3. Desain Mekanik

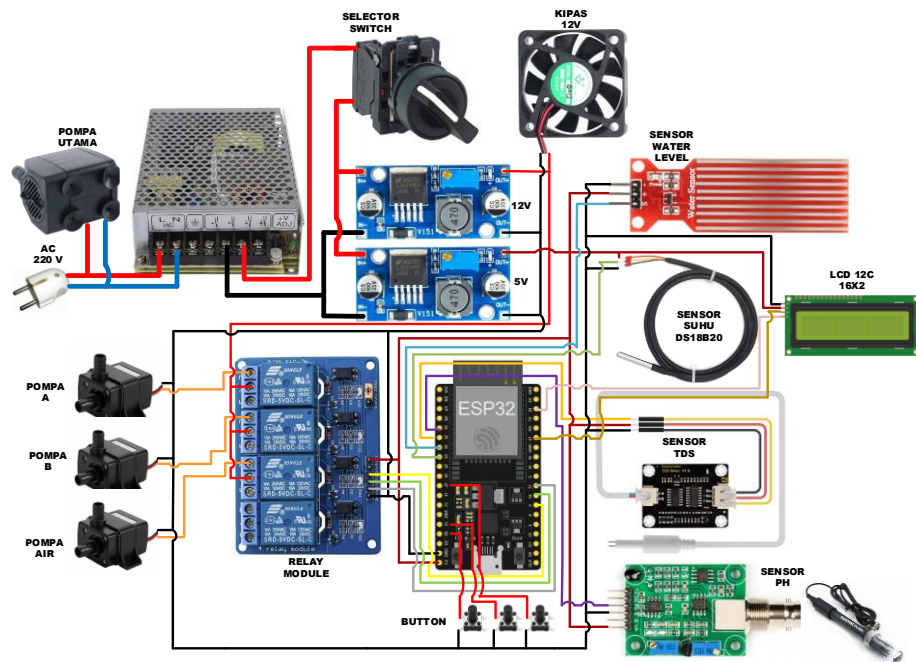
- **Sistem Tanaman:** Menggunakan NFT dengan pipa berlubang bertingkat. Larutan nutrisi mengalir tipis di dasar talang.
- **Media Tanam:** Menggunakan Netpot berisi rockwool yang dilengkapi sumbu dengan dasar netpot menyentuh aliran larutan pada talang.
- **Tandon Air:** Diletakkan disebelah samping tanaman untuk menampung aliran air yang mengalir.
- **Pompa Sirkulasi:** Pasang pompa celup didasar tandon dengan selang menuju talng bagian atas.
- **Sensor terendam:** pasang probe PH, TDS, dan suhu (DS18B20) ditandon.
- **Sensor Level:** pasasng dibagian tandon untuk mengukur volume air yang akan dialirkan.
- **Kotak kontrol (panel):** untuk menyimpan ESP32, relay, dan catu daya.

4. Skema Listrik & Wiring

4.1.Topologi Daya

- **Sumber:** Power Supplay 24 volt → buck pertama 12volt dan buck kedua 5 volt.
- **ESP:** ambil 5v kepin **VIN**(ESP32) → onboard regulator →3,3V
- **Mini fan:** mengambi daya dari 12 volt

4.2.Wiring Pin (ESP32)



Gambar 1. 1 Rangkaian Sistem

Tabel 1. 2 GPIO Mikrokontroler yang digunakan

Perangkat	Pin	ESP-32S	Catatan
Relay Pompa	Output	GPIO 4	Modul relay umumnya <i>active LOW</i>
Relay Nutrisi A	Output	GPIO 16	Pompa nutrisi A
Relay Nutrisi B	Output	GPIO 17	Pompa nutrisi B
Water Level	Input	GPIO 32	Pembacaan level air tandon
Sensor TDS	Input	GPIO 35	ADC1, <i>input only</i> , catu 3,3 V
Sensor PH	Input	GPIO 34	ADC1, <i>input only</i>
Sensor DS18B20	Input	GPIO 33	<i>Pull-up</i> 4,7k ke 3,3 V
Button_Air	Input	GPIO 27	INPUT_PULLUP, tombol ke GND.
Button_Nutrisi	Input	GPIO 2	Pin <i>strapping</i> .
Button_Layar	Input	GPIO 13	INPUT_PULLUP, tombol ke GND.
LCD_SCL	Output	GPIO 22	Jalur I2C <i>clock</i> .
LCD_SDA	Input/Output	GPIO 21	Jalur I2C data, dua arah.

5V	Power	Vin/5V	<i>Dari buck converter (relay dan LCD)</i>
3V3	Power	3,3V	Sensor yang membutuhkan daya 3,3V
GND	Power	Ground ESP32	<i>Common ground</i> untuk semua modul dan sensor.

5. Parameter dan Kalibrasi

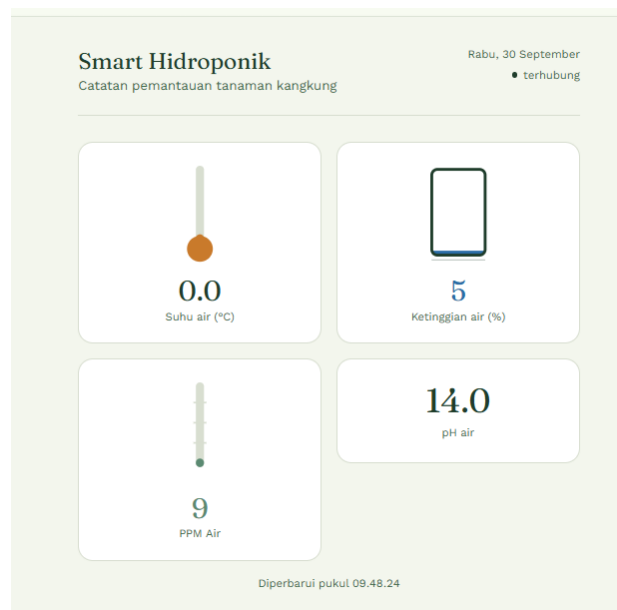
- **Pemberian Nutrisi tanaman hidroponik Kangkung dan Pakcoi**

Tabel 1. 3 Nutrisi Tanaman Kangkung dan Pakcoi

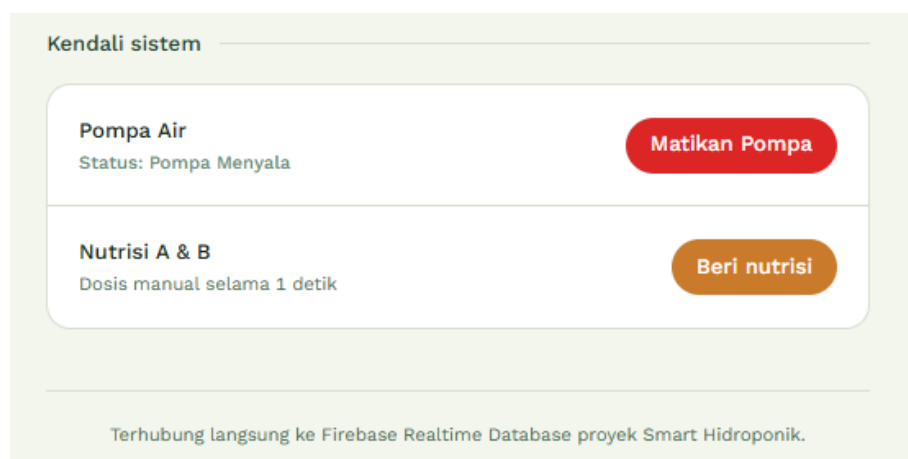
Minggu Ke-	Jenis Tanaman	
	Kangkung	Pakcoi
1	500 PPM	500 PPM
2	800 PPM	700 PPM
3	1200 PPM	900 PPM
4	1200 PPM	1200 PPM
5		1200 PPM
6-8		1300 PPM
PPM Maksimum	1050 PPM – 1400 PPM	1050 PPM – 1400 PPM
PH Air	5,5 – 6,5	7,0
Usia Panen	28 Hari	40 – 60 Hari

- Water level dengan nilai **80%**.
- Nutrisi a dan nutrisi b berbanding **sama**.

6. Website Monitoring



Gambar 1. 2 Website Monitoring



Gambar 1. 3 Website Kendali Sistem

Keterangan:

Gambar **pertama** merupakan bagian monitoring keadaan tanman hidroponik dengan kondisi mikrokontoler terhubung internet. Suhu air menerima nilai sensor DS18B20, ketinggian air menerima nilai dari sensor Water level, PPM air menerima nilai dari sensor TDS meter, dan PH menerima nilai dari sensor PH.

Gambar **Kedua** merupakan bagian Kendali sistem, untuk menyalakan pompa apabila tandon utama habis atau kurang dari batas dibawah **80%**. Untuk bagian memberikan nutrisi A dan Nutrisi B digunakan memberikan perintah menyalakan relay dan mengakibatkan pompa nutrisi a dan nutrisi b menyala secara Bersama.

Link akses website: <https://iotsmart.my.id/>

7. Mikrokontroler ESP32

Sourcode:

```
#include <Arduino.h>
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include <Preferences.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Firebase_ESP_Client.h>
#include <LiquidCrystal_I2C.h>
#include "addons/RTDBHelper.h"

// =====
// PIN HARDWARE SENSOR & RELAY
// =====

#define PPM_AIR 35
#define PH_AIR 34
#define SUHU_AIR 33
#define KETINGGIAN_AIR 32

#define Pompa_Air 4
#define Nutrisi_A 16
#define Nutrisi_B 17

#define btn_pompa 27
#define btn_nutrisi 2
#define btn_sw 13

// =====
// KONFIGURASI FIREBASE CREDENTIALS
// =====
#define API_KEY "AIzaSyDiCD3iClAS0leDZZUUKAB85HpCz7O9L_c"
#define DATABASE_URL "https://smart-hidroponik-26-default-rtdb.asia-southeast1.firebaseio.com"
#define USER_EMAIL "ESP32@smart-hidroponik-26.com"
#define USER_PASSWORD "BelajarHidroponik123///"

// WebServer & Preferences NVS
AsyncWebServer server(80);
Preferences preferences;

String ssid = "";
String password = "";

// Sensor, Display, & Firebase
OneWire oneWire(SUHU_AIR);
DallasTemperature DS18B20(&oneWire);
```

```

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;
FirebaseJson jsonSensors;

LiquidCrystal_I2C lcd(0x27, 16, 2);

// =====
// VARIABEL GLOBAL SHARED STATE (VOLATILE)
// =====
volatile bool statusPompa = false;
volatile bool triggerNutrisi = false;
volatile bool reqUpdatePompaFb = false;
volatile bool reqUpdateNutrisiFb = false;
volatile bool statusNutrisiTarget = false;
unsigned long waktuMulaiNutrisi = 0;

// 0: Monitoring PPM & pH, 1: Water Level & Suhu Air, 2: Info WiFi / AP Mode
volatile int lcdPage = 0;

// Status Koneksi & Jaringan (Non-Blocking State)
volatile bool isWifiConnected = false;
volatile bool isApModeActive = false;

// Variabel Global Pembacaan Sensor
volatile float g_t_air = 25.0;
volatile float g_h_air = 0.0;
volatile float g_d_ph = 0.0;
volatile float g_d_ppm = 0.0;

// =====
// TAMPILAN WEB SERVER HTML CONFIG
// =====
String getHTMLPage() {
    String html = "<!DOCTYPE HTML><html><head>";
    html += "<meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">";
    html += "<title>ESP32 WiFi Dashboard</title>";
    html += "<style>";
    html += "body { font-family: Arial, sans-serif; text-align: center; margin-top: 30px; background-color: #f0f2f5; }";
    html += ".card { background: white; max-width: 400px; margin: 0 auto; padding: 25px; border-radius: 12px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); }";
    html += ".status { font-weight: bold; padding: 10px; border-radius: 6px; margin-bottom: 20px; }";
    html += ".connected { background-color: #d4edda; color: #155724; }";
    html += ".ap-mode { background-color: #fff3cd; color: #856404; }";
    html += "input[type=text], input[type=password] { width: 100%; padding: 10px; margin: 8px 0; border: 1px solid #ccc; border-radius: 6px; box-sizing: border-box; }";
    html += ".pwd-container { position: relative; width: 100%; display: inline-block; }";
}

```

```

html += ".pwd-container input { padding-right: 40px; }";
html += ".toggle-btn { position: absolute; right: 10px; top: 50%; transform: translateY(-50%); cursor: pointer; user-select: none; font-size: 16px; }";
html += "input[type=submit], .btn { background-color: #007bff; color: white; padding: 10px 15px; border: none; border-radius: 6px; cursor: pointer; width: 100%; font-size: 15px; margin-top: 10px; }";
html += ".btn-danger { background-color: #dc3545; }";
html += "</style>";

// Script JavaScript untuk toggle visibilitas password
html += "<script>";
html += "function togglePassword() {";
html += "    var pwdInput = document.getElementById('passField');";
html += "    var toggleIcon = document.getElementById('toggleIcon');";
html += "    if (pwdInput.type === 'password') {";
html += "        pwdInput.type = 'text';";
html += "        toggleIcon.textContent = 'Hidden';";
html += "    } else {";
html += "        pwdInput.type = 'password';";
html += "        toggleIcon.textContent = 'Show';";
html += "    }";
html += "}";
html += "</script></head><body>";

html += "<div class=\"card\">";
html += "<h2>ESP32 WiFi Manager</h2>";

if (isWifiConnected) {
    html += "<div class=\"status connected\">Status: Terhubung ke WiFi</div>";
    html += "<p><b>SSID Terhubung:</b> " + WiFi.SSID() + "</p>";
    html += "<p><b>IP Address:</b> " + WiFi.localIP().toString() + "</p>";
    html += "<p><b>Sinyal (RSSI):</b> " + String(WiFi.RSSI()) + " dBm</p>";
} else {
    html += "<div class=\"status ap-mode\">Status: Access Point Mode</div>";
    html += "<p>ESP belum terhubung ke WiFi router.</p>";
}

html += "<hr><h3 style=\"margin-top:20px;\">Ganti / Atur WiFi</h3>";
html += "<form action=\"/save\" method=\"POST\">";
html += "<input type=\"text\" name=\"ssid\" placeholder=\"SSID WiFi\" required><br>";

// Input Password dengan Tombol Toggle (Mata)
html += "<div class=\"pwd-container\">";
html += "<input type=\"password\" id=\"passField\" name=\"password\" placeholder=\"Password WiFi\" required>";
html += "<span id=\"toggleIcon\" class=\"toggle-btn\" onclick=\"togglePassword()\">Show</span>";
html += "</div><br>";

html += "<input type=\"submit\" value=\"Simpan & Hubungkan\">";

```



```

html += "</form>";

if (isWifiConnected) {
    html += "<form action=\"/reset\" method=\"POST\" style=\"margin-top:10px;\">";
    html += "<button type=\"submit\" class=\"btn btn-danger\" onclick=\"return confirm('Yakin ingin mereset WiFi?')\">Reset Wifi</button>";
    html += "</form>";
}

html += "</div></body></html>";
return html;
}

// =====
// FUNGSI PEMBACAAN SENSOR ANALOG
// =====

double sensorPercent(uint8_t pin, int adcMin = 0, int adcMax = 4095) {
    const unsigned int sample = 10;
    long WLA = 0;
    for (int i = 0; i < sample; i++) {
        WLA += analogRead(pin);
        vTaskDelay(pdMS_TO_TICKS(1));
    }
    float WLAvg = (float)WLA / sample;
    if (WLAvg <= adcMin) return 0.0;
    if (WLAvg >= adcMax) return 100.0;
    return ((WLAvg - adcMin) / (float)(adcMax - adcMin)) * 100.0;
}

float getPH(uint8_t pin) {
    const float PH_SLOPE = -0.01333; // Sesuaikan dengan hasil kalibrasi 2 titik Anda
    const float PH_OFFSET = 29.98;
    const int sample = 50;
    int adcValues[sample];

    // 1. Ambil sampel ADC
    for (int i = 0; i < sample; i++) {
        adcValues[i] = analogRead(pin);
        vTaskDelay(pdMS_TO_TICKS(5)); // Atau delay(5); jika tanpa FreeRTOS
    }

    // 2. Urutkan nilai ADC dari terkecil ke terbesar (Bubble Sort sederhana)
    for (int i = 0; i < sample - 1; i++) {
        for (int j = i + 1; j < sample; j++) {
            if (adcValues[i] > adcValues[j]) {
                int temp = adcValues[i];
                adcValues[i] = adcValues[j];
                adcValues[j] = temp;
            }
        }
    }
}

```

```

// 3. Buang 3 nilai terendah dan 3 nilai tertinggi (abaikan noise ekstrem)
long totalAdc = 0;
int validSamples = 0;
for (int i = 3; i < sample - 3; i++) {
    totalAdc += adcValues[i];
    validSamples++;
}

float rataAdc = (float)totalAdc / validSamples;
float nilaiPH = (rataAdc * PH_SLOPE) + PH_OFFSET;

if (nilaiPH < 0.0) nilaiPH = 0.0;
if (nilaiPH > 14.0) nilaiPH = 14.0;

return nilaiPH;
}

float getPPM(uint8_t pin, float suhu) {
    const unsigned int sample = 15;
    long TDSA = 0;
    for (int i = 0; i < sample; i++) {
        TDSA += analogRead(pin);
        vTaskDelay(pdMS_TO_TICKS(1));
    }
    float TDSAvg = (float)TDSA / sample;
    float TDSV = (TDSAvg / 4095.0) * 3.3;
    if (suhu <= 0 || suhu >= 100) suhu = 25.0;

    float CV = TDSV / (1.0 + 0.02 * (suhu - 25.0));
    return ((133.42 * pow(CV, 3) - 255.86 * pow(CV, 2) + 857.39 * CV) * 0.5) * 1.486 +
    8.9586;
}

// =====
// TASK 1 (CORE 1): KONTROL RELAY & TOMBOL
// =====

void TaskControl(void *pvParameters) {
    bool lastBtnPompaState = LOW;
    bool lastBtnNutrisiState = LOW;
    bool lastBtnSwState = LOW;

    vTaskDelay(pdMS_TO_TICKS(500));

    digitalWrite(Pompa_Air, statusPompa ? LOW : HIGH);
    digitalWrite(Nutrisi_A, HIGH);
    digitalWrite(Nutrisi_B, HIGH);

    for (;;) {
        // 1. Tombol Pompa Air (Ditekan = HIGH)
        bool currentBtnPompa = digitalRead(btn_pompa);

```

```

    if (currentBtnPompa == HIGH && lastBtnPompaState == LOW) {
        statusPompa = !statusPompa;
        digitalWrite(Pompa_Air, statusPompa ? LOW : HIGH);
        reqUpdatePompaFb = true;
        vTaskDelay(pdMS_TO_TICKS(200));
    }
    lastBtnPompaState = currentBtnPompa;

    // 2. Tombol Nutrisi (Ditekan = HIGH)
    bool currentBtnNutrisi = digitalRead(btn_nutrisi);
    if (currentBtnNutrisi == HIGH && lastBtnNutrisiState == LOW && !triggerNutrisi)
    {
        triggerNutrisi = true;
        waktuMulaiNutrisi = millis();

        digitalWrite(Nutrisi_A, LOW);
        digitalWrite(Nutrisi_B, LOW);

        reqUpdateNutrisiFb = true;
        statusNutrisiTarget = true;
        vTaskDelay(pdMS_TO_TICKS(200));
    }
    lastBtnNutrisiState = currentBtnNutrisi;

    // 3. Tombol Switch Display LCD (Ditekan = HIGH) -> Rotasi 3 Halaman (0, 1, 2)
    bool currentBtnSw = digitalRead(btn_sw);
    if (currentBtnSw == HIGH && lastBtnSwState == LOW) {
        lcdPage = (lcdPage + 1) % 3;
        lcd.clear();
        vTaskDelay(pdMS_TO_TICKS(200));
    }
    lastBtnSwState = currentBtnSw;

    // 4. Auto-Off Timer Nutrisi (1 Detik Timeout)
    if (triggerNutrisi && (millis() - waktuMulaiNutrisi >= 1000)) {
        digitalWrite(Nutrisi_A, HIGH);
        digitalWrite(Nutrisi_B, HIGH);
        triggerNutrisi = false;

        reqUpdateNutrisiFb = true;
        statusNutrisiTarget = false;
    }

    vTaskDelay(pdMS_TO_TICKS(20));
}

// =====
// TASK 2 (CORE 0): KONEKSI WIFI & NETWORK MANAGEMENT
// =====
void TaskNetworkManager(void *pvParameters) {

```

```

preferences.begin("wifi-config", false);
ssid = preferences.getString("ssid", "");
password = preferences.getString("password", "");

if (ssid != "") {
    WiFi.mode(WIFI_STA);
    WiFi.disconnect(true);
    vTaskDelay(pdMS_TO_TICKS(100));

    WiFi.begin(ssid.c_str(), password.c_str());
    Serial.print("Mencoba koneksi ke ");
    Serial.println(ssid);

    int timeout = 0;
    while (WiFi.status() != WL_CONNECTED && timeout < 20) {
        vTaskDelay(pdMS_TO_TICKS(500));
        Serial.print(".");
        timeout++;
    }
    Serial.println();
}

if (WiFi.status() == WL_CONNECTED) {
    isConnected = true;
    isApModeActive = false;
    Serial.println("Terhubung ke WiFi!");
    Serial.print("IP Local: ");
    Serial.println(WiFi.localIP());

    config.api_key = API_KEY;
    config.database_url = DATABASE_URL;
    auth.user.email = USER_EMAIL;
    auth.user.password = USER_PASSWORD;

    Firebase.begin(&config, &auth);
    Firebase.reconnectWiFi(false);
    fbdo.setResponseSize(2048);
} else {
    isConnected = false;
    isApModeActive = true;
    WiFi.disconnect(true);
    vTaskDelay(pdMS_TO_TICKS(100));
    WiFi.mode(WIFI_AP);
    WiFi.softAP("ESP-Config-WiFi", "12345678");
    Serial.println("Mengaktifkan Mode AP Murni...");
    Serial.print("IP SoftAP: ");
    Serial.println(WiFi.softAPIP());
}

server.on("/", HTTP_GET, [] (AsyncWebServerRequest *request){
    request->send(200, "text/html", getHTMLPage());
});

```

```

});

server.on("/save", HTTP_POST, [] (AsyncWebServerRequest *request){
    if (request->hasParam("ssid", true) && request->hasParam("password", true)) {
        preferences.putString("ssid", request->getParam("ssid", true)->value());
        preferences.putString("password", request->getParam("password", true)-
>value());
        request->send(200, "text/html", "<h3>Konfigurasi Disimpan! Merestart...</h3>");
        delay(2000);
        ESP.restart();
    } else {
        request->send(400, "text/plain", "Form tidak lengkap!");
    }
});

server.on("/reset", HTTP_POST, [] (AsyncWebServerRequest *request){
    preferences.clear();
    request->send(200, "text/html", "<h3>WiFi Dihapus! Merestart ke AP Mode...</h3>");
    delay(2000);
    ESP.restart();
});

server.begin();

for (;;) {
    isWifiConnected = (WiFi.status() == WL_CONNECTED);

    if (isWifiConnected && Firebase.ready()) {
        if (reqUpdatePompaFb) {
            Firebase.RTDB.setBool(&fbdo, "/kontrol/air", statusPompa);
            reqUpdatePompaFb = false;
        }

        if (reqUpdateNutrisiFb) {
            if (Firebase.RTDB.setBool(&fbdo, "/kontrol/Nutrisi", statusNutrisiTarget)) {
                reqUpdateNutrisiFb = false;
            }
        }

        jsonSensors.clear();
        jsonSensors.add("suhu_air", g_t_air);
        jsonSensors.add("ketinggian_air", g_h_air);
        jsonSensors.add("PPM", g_d_ppm);
        jsonSensors.add("PH", g_d_ph);
        Firebase.RTDB.updateNode(&fbdo, "/monitoring/", &jsonSensors);

        if (Firebase.RTDB.getBool(&fbdo, "/kontrol/air")) {
            bool fbStatusAir = fbdo.boolData();
            if (fbStatusAir != statusPompa) {
                statusPompa = fbStatusAir;
                digitalWrite(Pompa_Air, (statusPompa && g_h_air < 80.0) ? LOW : HIGH);
            }
        }
    }
}

```

```

    }
}

if (!reqUpdateNutrisiFb && !triggerNutrisi) {
    if (Firebase.RTDB.getBool(&fbdo, "/kontrol/Nutrisi")) {
        bool fbStatusNutrisi = fbdo.boolData();
        if (fbStatusNutrisi) {
            triggerNutrisi = true;
            waktuMulaiNutrisi = millis();
            digitalWrite(Nutrisi_A, LOW);
            digitalWrite(Nutrisi_B, LOW);
        }
    }
}
} else {
    reqUpdatePompaFb = false;
    reqUpdateNutrisiFb = false;
}

vTaskDelay(pdMS_TO_TICKS(1000));
}
}

// =====
// TASK 3 (CORE 0): PEMBACAAN SENSOR & DISPLAY LCD
// =====

void TaskMainSystem(void *pvParameters) {
    unsigned long lastTempRequest = 0;

    for (;;) {
        if (millis() - lastTempRequest >= 1000) {
            g_t_air = DS18B20.getTempCByIndex(0);
            if(g_t_air <= 0) g_t_air = 0;
            DS18B20.requestTemperatures();
            lastTempRequest = millis();
        }

        g_h_air = sensorPercent(KETINGGIAN_AIR);
        g_d_ph = getPH(PH_AIR);
        g_d_ppm = getPPM(PPM_AIR, g_t_air);

        char line1[17];
        char line2[17];

        // Halaman 0: Monitoring PPM & pH
        if (lcdPage == 0) {
            snprintf(line1, sizeof(line1), "PPM: %-11.1f", g_d_ppm);
            snprintf(line2, sizeof(line2), "PH : %-11.2f", g_d_ph);
        }

        // Halaman 1: Monitoring Water Level & Suhu Air
        else if (lcdPage == 1) {

```

```

        snprintf(line1, sizeof(line1), "% Air: %.1f%%", g_h_air);
        snprintf(line2, sizeof(line2), "Suhu: %9.1f C", g_t_air);
    }
    // Halaman 2: Information Network
    else if (lcdPage == 2) {
        if (isWifiConnected) {
            String wifiSsid = WiFi.SSID();
            if (wifiSsid.length() > 11) wifiSsid = wifiSsid.substring(0, 11);
            snprintf(line1, sizeof(line1), "WIFI:%-11s", wifiSsid.c_str());
            snprintf(line2, sizeof(line2), "%-16s", WiFi.localIP().toString().c_str());
        } else {
            snprintf(line1, sizeof(line1), "MODE: AP CONFIG");
            snprintf(line2, sizeof(line2), "%-16s", WiFi.softAPIP().toString().c_str());
        }
    }
}

lcd.setCursor(0, 0);
lcd.print(line1);
lcd.setCursor(0, 1);
lcd.print(line2);

vTaskDelay(pdMS_TO_TICKS(500));
}
}

// =====
// SETUP UTAMA
// =====
void setup() {
    Serial.begin(115200);

    lcd.init();
    lcd.backlight();
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("System Starting..");

    digitalWrite(Pompa_Air, HIGH);
    digitalWrite(Nutrisi_A, HIGH);
    digitalWrite(Nutrisi_B, HIGH);

    pinMode(Pompa_Air, OUTPUT);
    pinMode(Nutrisi_A, OUTPUT);
    pinMode(Nutrisi_B, OUTPUT);

    digitalWrite(Pompa_Air, HIGH);
    digitalWrite(Nutrisi_A, HIGH);
    digitalWrite(Nutrisi_B, HIGH);

    pinMode(btn_pompa, INPUT_PULLDOWN);
    pinMode(btn_nutrisi, INPUT_PULLDOWN);
}

```

```

pinMode(btn_sw, INPUT_PULLDOWN);

DS18B20.begin();
DS18B20.waitForConversion(false);
DS18B20.requestTemperatures();

// FREERTOS TASKS CREATION
xTaskCreatePinnedToCore(
    TaskControl,
    "TaskControl",
    4096,
    NULL,
    2,
    NULL,
    1 // Core 1
);

xTaskCreatePinnedToCore(
    TaskMainSystem,
    "TaskMainSystem",
    4096,
    NULL,
    1,
    NULL,
    0 // Core 0
);

xTaskCreatePinnedToCore(
    TaskNetworkManager,
    "TaskNetworkManager",
    8192,
    NULL,
    1,
    NULL,
    0 // Core 0
);
}

void loop() {
    vTaskDelete(NULL);
}

```

8. Kendala dan Keamanan

- Menggunakan **firebase** dikarenakan memiliki fitur cepat dalam menyimpan data.
- **Water level** ketika ditempatkan pada tandon wajib dilapisi bagian pin agar tidak terkena air.
- **Sumber arus AC** harus jauh dari air agar tidak korsleting.

- Terdapat **saklar On/Off** sistem diluar panel dengan fungsi memutuskan sistem digunakan apabila terdapat masalah muncul.

9. Dokumentasi Pemasangan:

